

Testen von Methoden

Lernziele

- Implementierung von Methoden nach vorgegebenen Schnittstellen
- Definieren und Ausführen von Schnittstellentests

Hintergrund

Die sogenannten „**Unit Tests**“ in der Software-Entwicklung sind automatisierte Schnittstellentests, d.h. jede einzelne Methode eines Programms wird in Bezug auf ihre Eingangsparameter und den Rückgabewert automatisch auf Herz und Nieren geprüft.

Der „**Test Driven Development**“ Ansatz geht sogar soweit, dass zuallererst die Testfälle einer Methode definiert werden, bevor man sich an die Implementierung der Methode selbst macht. Also sind

- zu Beginn der Implementierung alle Testfälle auf „**rot**“ (= fehlgeschlagen, „failed“),
- und nach der Fertigstellung der Implementierung müssen alle Testfälle „**grün**“ (= erfolgreich, „passed“) sein.

Sobald alle Testfälle grün sind, werden die neuen Tests in die täglichen automatischen „**Regressionstests**“ aufgenommen, hier wird überprüft, ob nach allfälligen Programmänderungen noch immer alle Testfälle erfolgreich durchlaufen, oder ob sich Verschlechterungen („Regressionen“) eingeschlichen haben.

Aufgabenstellung

Heute bekommen Sie eine Visual Studio Projektvorlage, in der eine Testfunktion namens „**Assert**“ zur Verfügung steht, die zum Testen von Methodenschnittstellen verwendet werden soll.

Am Ende der Tests kann mit „**PrintSummary**“ eine Statistik über alle erfolgreichen („passed“) und fehlgeschlagenen („failed“) Testfälle ausgegeben werden.

Aufgabe 1

- Implementieren Sie alle Methoden, die bereits im Visual Studio Projekt definiert wurden, und sorgen Sie dafür, dass alle Testfälle in der Main()-Methode „grün“ werden. Also:
 - **GetSplittingLine**
 - **Compare**
 - **IsElementOf**
 - **GetMaximum**
- Erweitern Sie die Testfälle, sodass auch Randbereiche oder ungültige Eingangsparameter überprüft werden. Zum Beispiel muss „GetMaximum“ auch für ein Array, das nur negative Zahlen enthält, funktionieren!

Aufgabe 2

Fügen Sie Testfälle und Implementierungen für folgende Methoden hinzu:

- **SortCharacters:** Diese Methode bekommt einen String als Parameter und liefert einen neuen String als Rückgabewert, der alle Zeichen im übergebenen String in sortierter Reihenfolge enthält. Beispiel:
 - `Assert(SortCharacters("dacgf") == "acdfg",
"Sortierung von ,dacgf` ergibt ,acdfg`");`
- **ToUpper:** Diese Methode bekommt einen String als Parameter und liefert einen neuen String als Rückgabewert, der alle enthaltenen Buchstaben auf Großbuchstaben konvertiert enthält.
- **ToLower:** Diese Methode bekommt einen String als Parameter und liefert einen neuen String als Rückgabewert, der alle enthaltenen Buchstaben auf Kleinbuchstaben konvertiert enthält.
- **ShiftCharacters:** Diese Methode bekommt einen String und einen Integer als Parameter und soll alle Zeichen um den angegebenen Betrag verschieben. Beispiel:
 - `Assert(ShiftCharacters("abcdef", 1) == "bcdefg",
"Verschiebung von ,abcdef` mit 1 ergibt ,bcdefg`")`
 - `Assert(ShiftCharacters("bcdefg", -1) == "abcdef",
"Verschiebung von ,bcdefg` mit -1 ergibt ,abcdef`")`
- **Substring:** Diese Methode erhält einen String als Parameter, einen Integer-Parameter für die Position, ab der ein Teilstring entnommen werden soll, sowie die Länge des gesuchten Teilstrings. Rückgabewert ist der Teilstring. Beispiel:
 - `Assert(Substring("abcdef", 3, 2) == "de", "Teilstring der Länge 2 ab
Position 3")`
 - `Assert(Substring("abcdef", 5, 3) == "f", "Teilstring der Länge 3 ab
Position 5")`
- **Sum:** Diese Methode erhält ein Integer-Array als Parameter, und liefert einen Integer-Wert zurück, der die Summe aller Elemente des Integer-Arrays ist. Beispiel:
 - `Assert(Sum(new int[]{1, 2, 3, 4}) == 10, "Summe aus 1+2+3+4 ist 10")`
- **SumOfProducts:** Diese Methode erhält zwei Integer-Arrays als Parameter, und liefert die Summe der Produkte der einzelnen Elemente zurück. Beispiel:
 - `Assert(SumOfProducts (new int[]{1, 2, 3, 4}, new int[]{2, 2, 2, 2}) ==
10, "Produktsumme aus 1*2+2*2+3*2+4*2 ist 20")`

Hinweise:

- Überlegen Sie bei jeder Methode genau, was im Fall von ungültigen Parameterübergaben passieren soll!
- Wenn zum Beispiel die beiden Arrays für SumOfProducts verschieden lang sind, sollen die fehlenden Stellen durch die „0“ ersetzt werden:
 - `Assert(SumOfProducts (new int[]{1, 2, 3, 4}, new int[]{2, 2, 2}) == 10,
"Produktsumme aus 1*2+2*2+3*2+4*0 ist 12")`
- Überprüfen Sie auch bei jeder Methode, ob tatsächlich ein String-Objekt oder Integer-Array übergeben wurde, oder ob der Parameter-Wert „null“ ist!